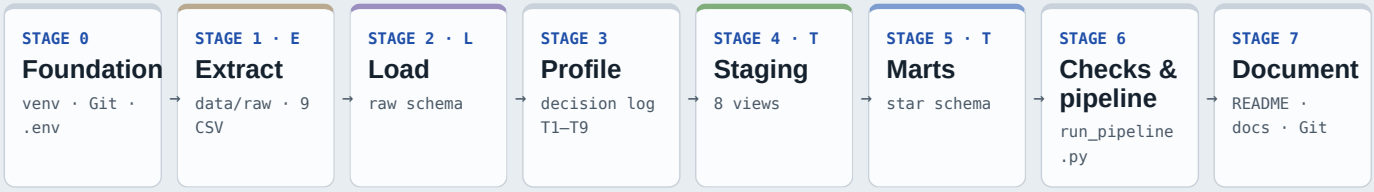
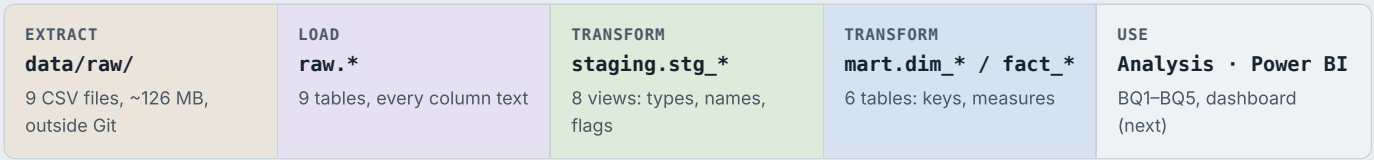
OLIST COMMERCIAL ANALYTICS · ETL STUDY MAP · WEEKS 1–2

# Olist ETL Process Map

How 9 CSV files became a tested star schema in PostgreSQL, step by step: what each stage does, the code and tools used, the problems met along the way and how each was fixed. Click a stage to jump to its section.

**1,550,622** raw rows **9** raw tables **8** staging views **6** mart tables **26** automated checks **1** command: `py run_pipeline.py`




The pattern is **ELT**: data is loaded into PostgreSQL unchanged and transformed there with SQL. "ETL" is the common name for the whole process.

## STAGE 0 Foundation: a reproducible workspace

Before any data moves: the same tools, the same package versions, passwords kept out of code, and every change recorded in version control.

### INPUTS → OUTPUTS

in — An empty folder on drive D

out — `.venv`, `requirements.txt`, folder scaffold, `.gitignore`, `.env`, GitHub repo `dbsaragih/olist-commercial-analytics`

### TOOLS

`PowerShell` `Python 3.14 (py)` `venv · pip` `Git 2.56` `GitHub` `VS Code`

### KEY CONCEPTS

- **Virtual environment:** a private Python per project, active per window. `(.venv)` must show in the prompt.
- **requirements.txt** rebuilds the exact package set.
- **.gitignore** keeps data, secrets and editor files out of Git.
- **Absolute vs relative paths:** a relative path means "in the current folder."

### CODE

```
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install pandas sqlalchemy "psycopg[binary]" python-dotenv matplotlib jupyter
pip freeze | Out-File -Encoding utf8 requirements.txt
git init -b main
git remote add origin https://github.com/dbsaragih/olist-commercial-analytics.git
git push -u origin main
```

### .GITIGNORE (CORE LINES)

```
.venv/
.env
data/raw/*
data/interim/*
data/processed/*
.vscode/
```

### OBSTACLES AND FIXES 5

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| `git` not recognised | PowerShell window opened before Git was on the PATH | Closed and reopened PowerShell | A window reads the PATH only when it starts |
| Git listed thousands of `.cache/…` files | `git init` ran in `C:\Users\Lenovo` | Checked the creation time, removed `C:\Users\Lenovo\.git`, re-initialised in the project | Check the prompt before every Git command |

| | | | |
|---|---|---|---|
| `.gitignore` missing from the project | Saved with a relative path while PowerShell was in the user folder | Created it with the full path | Use absolute paths when creating files from the terminal |
| `.vscode/` still untracked after adding the rule | Leading spaces before the pattern | Removed the spaces | Invisible characters matter in config files |
| `term not recognised` for `.venv/` and `To` | File contents and expected output were pasted into PowerShell | Ran only the command lines | Separate commands from expected output |

**Evidence:** `git status` shows a clean tree; `git check-ignore -v .env` confirms the password file is ignored.

## STAGE 1 · E Extract: get the source data unchanged

Download the client's data exactly as delivered and keep that copy untouched. Every later step can be rerun from it.

### INPUTS → OUTPUTS

in Kaggle: Brazilian E-Commerce Public Dataset by Olist (ZIP)

out 9 CSV files (~126 MB) directly in `data/raw/`

### TOOLS

`Browser` `File Explorer` `PowerShell`

### KEY CONCEPTS

- **Raw files are never edited.** Cleaning happens later, in code.
- **Data stays out of Git** (`data/raw/*`): too large, and the README says where to download it.

### CODE

```
dir data\raw      # expect 9 CSV files
git status        # expect "working tree clean": data is ignored
```

### OBSTACLES AND FIXES 1

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| Risk of files landing in a subfolder | Windows "Extract All" can create an extra folder | Extracted straight into `data\raw` and checked with `dir` | Verify the location before building on it |

**Evidence:** 9 files listed; Git reports nothing new after adding ~126 MB.

## STAGE 2 · L Load: copy the files into PostgreSQL

Put each CSV into the database as it is, then prove that every row arrived.

**INPUTS → OUTPUTS**

in 9 CSV files

out `raw` schema: 9 tables, 1,550,622 rows, every column text

file `src/load_raw.py`

**TOOLS**

Python · pandas · SQLAlchemy · psycopg 3 · python-dotenv · PostgreSQL 18

**KEY CONCEPTS**

- **Load as text** (`dtype=str`): no silent conversion; zip `01310` keeps its zero.
- **Rerunnable:** `if_exists="replace"` never duplicates rows.
- **Secrets in `.env`:** the password is never in code.
- **Load validation:** rows in file = rows in table, or the run stops.

**CODE · SRC/LOAD_RAW.PY (CORE)**

```
df = pd.read_csv(RAW_DIR / file_name, dtype=str)   # every
column as text
df.to_sql(table, engine, schema="raw",
if_exists="replace",
          index=False, chunksize=10_000)           #
rerunnable

db_rows = conn.execute(
    text(f"SELECT COUNT(*) FROM raw.
{table}")).scalar_one()
status = "OK" if db_rows == len(df) else "MISMATCH"
...
if mismatches:                                     # stop
the pipeline
    raise RuntimeError(f"Row count mismatch in: {',
'.join(mismatches)}")
```

**.ENV (NEVER COMMITTED)**

```
PGHOST=localhost
PGPORT=5432
PGUSER=postgres
PGPASSWORD=********
PGDATABASE=olist
```

**OBSTACLES AND FIXES** 3

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| Password would be published on GitHub | Connection details written in code | `.env` file, ignored by Git, read with python-dotenv | Verify with `git check-ignore -v .env` before any push |
| A MISMATCH would be printed but the run would carry on | The loader only printed its check | Raise an error when any table mismatches | A check that doesn't stop the run is only a log line |
| Reloading raw would fail: "cannot drop table raw.orders because other objects depend on it" | Staging views are built on the raw tables | The pipeline drops the staging schema before reloading | Clear dependents before replacing what they depend on |

**Evidence:** 9 of 9 tables OK, 1,550,622 rows (geolocation alone is 1,000,163).

## STAGE 3 Profile: learn the data before changing it

Check keys, missing values, date ranges and links between tables, then decide how each problem will be treated. The result is the decision log (T1–T9) that the transform stages apply.

### INPUTS → OUTPUTS

in `raw` schema

out `sql/01–03_profile_*.sql`, `docs/02_data_quality_findings.md` (decision log, materiality threshold)

### TOOLS

SQL · DBeaver · VS Code (notes) · Markdown

### KEY CONCEPTS

- **Profiling checklist:** rows, keys, nulls, values, dates, logic, links.
- **Reconciliation:** two routes to the same number must agree.
- **Materiality:** <1% exclude or document, 1–5% label, >5% fix; then check bias and sensitivity.
- **Data issue vs business insight;** business event vs data artefact.

### CODE · PROFILING PATTERNS

```
-- Nulls: COUNT(*) counts rows, COUNT(col) skips NULLs
SELECT COUNT(*) - COUNT(order_delivered_customer_date)
FROM raw.orders;

-- Orphans with an anti-join
SELECT COUNT(*) FROM raw.orders o
WHERE NOT EXISTS (SELECT 1 FROM raw.order_items i
                  WHERE i.order_id = o.order_id);

-- Real date coverage (::timestamp, never ::timestamptz)
SELECT DATE_TRUNC('month',
order_purchase_timestamp::timestamp)::date,
       COUNT(*)
FROM raw.orders GROUP BY 1 ORDER BY 1;
```

### OBSTACLES AND FIXES 5

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| Dates displayed with `+0700` | `::timestamptz` applied the Jakarta session time zone | `::timestamp` (Olist times carry no zone) | Choose the type that matches the data |
| `SET TIME ZONE` added as a fix | Treated the symptom; changes the whole session | Removed; fixed the cast instead | Fix the cause, not the display |
| `order_item_id` sorted 1, 10, 11, 2 | Stored as text in raw | `::int` (in staging) | Text sorts as text |
| Orders fall to 16 and 4 in Sep–Oct 2018 | Extract cut-off, not a collapse | Analysis window Jan 2017 – Aug 2018 (T1) | Ask "business event or data artefact?" |
| Findings first too absolute, then too hedged | Describing results instead of deciding | What / Why / So what, then pyramid style for the client | Every finding ends in a decision or a link to a business question |

**Evidence:** reconciliations closed exactly: 2,965 null delivery dates; 775 orders without items; 3,345 extra orders per person; 768 orders without review.

## STAGE 4 · T Staging: clean, typed views

One view per source table: correct types, consistent names, and the decision log applied as flags and labels. No rows deleted, no business calculations.

**INPUTS → OUTPUTS**

| | |
|---|---|
| in | `raw` schema |
| out | `staging`: `stg_orders`, `stg_order_items`, `stg_order_reviews`, `stg_customers`, `stg_sellers`, `stg_products`, `stg_category_translation`, `stg_state_region` |
| files | `sql/staging/*.sql`, `tests/test_stg_*.sql` |

**TOOLS**

`SQL views` `VS Code + PostgreSQL extension`

**KEY CONCEPTS**

- **Views vs tables:** a view stores the query and is always in sync.
- **Idempotent:** `DROP VIEW IF EXISTS … CASCADE` then `CREATE VIEW`.
- **Flags, not deletion;** fail loudly rather than filter silently.
- **Deterministic dedupe:** `ROW_NUMBER()` with tie-breakers and `NULLS LAST`.
- **Identifiers stay text;** manual mappings with `UNION ALL` + `VALUES`.

**CODE · SQL/STAGING/STG_ORDERS.SQL (EXCERPT)**

```
DROP VIEW IF EXISTS staging.stg_orders CASCADE;

CREATE VIEW staging.stg_orders AS
SELECT
    order_id,
    order_purchase_timestamp::timestamp     AS
purchase_ts,
    (order_status = 'delivered')            AS
is_delivered,
    (order_purchase_timestamp::timestamp >= '2017-01-01'
     AND order_purchase_timestamp::timestamp < '2018-09-
01')
                                            AS
in_analysis_period,  -- T1
    COALESCE(order_delivered_carrier_date::timestamp
             < order_purchase_timestamp::timestamp, FALSE)
                                            AS
excl_carrier_leg     -- T4
FROM raw.orders;
```

**CODE · STG_ORDER_REVIEWS.SQL (T6)**

```
ROW_NUMBER() OVER (
    PARTITION BY order_id
    ORDER BY review_answer_timestamp::timestamp DESC NULLS
LAST,
             review_creation_date::timestamp    DESC NULLS
LAST,
             review_id                          DESC
) = 1                                       AS
is_latest_for_order
```

**OBSTACLES AND FIXES** 4

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| VS Code: `relation "staging.stg_orders" does not exist` | View not yet created, or held in an uncommitted DBeaver transaction | Built the view and confirmed it with `information_schema.views` | Other connections only see committed changes |
| First experiment view filtered rows with `WHERE` | Bad values would vanish silently | No filter; a bad value now stops the build | Fail loudly beats dropping silently |
| T5 needed the purchase date from another table | Cross-table logic in staging breaks the one-to-one rule and creates CASCADE dependencies | Moved T5 to `fact_order_items` | Staging maps one source table; joins belong in marts |
| Exploration query saved inside a test file | Quick look typed into the wrong file | `git restore`; exploration goes in `_notes\scratch.sql` | Tests contain only checks with expected values |

**Evidence:** every staging test matched profiling (99,441 orders; 96,478 delivered; 99,092 in period; T3 = 8; T4 = 166); 0 products without an English name; 23,995 zip codes kept their leading zero.

## STAGE 5 · T Marts: the star schema

Stored tables built for analysis and Power BI: four dimensions to slice by, two fact tables to measure, with keys the database enforces.

INPUTS → OUTPUTS

in `staging` views

out `mart`: `dim_date`, `dim_customer`, `dim_product`, `dim_seller`, `fact_order_items` (one row per item), `fact_orders` (one row per order)

files `sql/marts/*.sql`, `tests/test_mart_*.sql`, `docs/03_data_model.md`

TOOLS

`SQL (CREATE TABLE AS)` `Primary and foreign keys` `Mermaid`

KEY CONCEPTS

- **Grain first:** one row = one item, or one order. Wrong grain double-counts.
- **Keys as build-time tests:** duplicates or orphans stop the build.
- **Natural keys** plus a `YYYYMMDD` date key; full-year date table.
- **Point-in-time state** on the fact; **role-playing** date dimension.
- **Additive vs non-additive:** never average `freight_share`. **NULL, not zero,** for undelivered orders.

CODE · SQL/MARTS/DIM_SELLER.SQL

```
DROP TABLE IF EXISTS mart.dim_seller CASCADE;

CREATE TABLE mart.dim_seller AS
SELECT s.seller_id, s.zip_code_prefix, s.city, s.state,
r.region
FROM staging.stg_sellers s
LEFT JOIN staging.stg_state_region r ON r.state = s.state;

ALTER TABLE mart.dim_seller ADD PRIMARY KEY (seller_id);
```

CODE · FACT_ORDER_ITEMS.SQL (KEYS, T5)

```
(i.shipping_limit_ts > o.purchase_ts + INTERVAL '30 days')
                                        AS excl_handover
-- T5
...
ALTER TABLE mart.fact_order_items
  ADD PRIMARY KEY (order_id, order_item_id);
ALTER TABLE mart.fact_order_items
  ADD CONSTRAINT fk_foi_product FOREIGN KEY (product_id)
      REFERENCES mart.dim_product (product_id);
```

OBSTACLES AND FIXES 2

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| Data model diagram had no `delivered_date_key` | Diagram drawn with selected columns only | Documented the role-playing date dimension; redrew the diagram from the built tables | When build and docs disagree, fix the docs that week |
| Test expected 96,470, got 96,476 | Test counted all orders; 6 non-delivered orders have a delivery date (finding O6) | Scoped the test by `is_delivered` | When a test fails, check the test as carefully as the data |

**Evidence:** GMV of R$ 13,591,643.70 is identical in staging, `fact_order_items` and `fact_orders`; delivered orders in the analysis period: 96,211, GMV R$ 13.18 M.

## STAGE 6 Checks and pipeline: one command

A Python orchestrator runs every step in dependency order, then 26 checks compare actual values with verified ones. Any failure stops the run with exit code 1.

INPUTS → OUTPUTS

in All scripts above

out `run_pipeline.py`, `tests/pipeline_checks.sql`

TOOLS

`Python` `psycopg 3` `argparse` `PowerShell`

KEY CONCEPTS

- **Orchestration:** build order written once, in `BUILD_STEPS`.
- **Dependencies run both ways:** build bottom-up, clear dependents first.
- **Regression checks:** expected vs actual, machine-readable.
- **Exit codes:** 0 = success, 1 = failure.
- `--skip-load` rebuilds SQL layers only (about 20 s).

CODE · RUN_PIPELINE.PY (CORE)

```
if not args.skip_load:
    load_raw(conn)            # drops staging, reloads raw

for relative_path in BUILD_STEPS:
    run_sql_file(conn, relative_path)

failures = run_checks(conn)   # expected vs actual
if failures:
    sys.exit(1)
```

CODE · TESTS/PIPELINE_CHECKS.SQL (ONE ROW)

```
( 2, 'staging', 'delivered orders', 96478,
     (SELECT COUNT(*) FROM staging.stg_orders WHERE
is_delivered)),
```

OBSTACLES AND FIXES 2

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| Manual tests needed a person to read comments | Expected values lived only in comments | One checks file in a fixed shape: name, expected, actual | Automate what you verified once |
| Raw reload blocked by dependent views | See Stage 2 | `DROP SCHEMA IF EXISTS staging CASCADE` before loading | Order of operations follows dependencies |

**Evidence:** 26 of 26 checks PASS; `PIPELINE OK` in 21 s with `--skip-load`; the full run from the CSV files also passes.

## STAGE 7 Document and version

Make the work reviewable: a commit for each logical change, documents that match what was built, and a README that tells a recruiter what exists.

### OUTPUTS

out `README.md`, `docs/01` problem statement, `02` data quality findings, `03` data model, `04` ETL pipeline; clean Git history

### TOOLS

`Git` `GitHub` `Markdown` `Mermaid`

### KEY CONCEPTS

- **Pyramid style:** the conclusion first, then evidence.
- **Claim only what exists:** "automated" only once it was.
- **One commit per change,** with a message that says what changed.

### CODE · COMMIT ROUTINE

```
git status                  # 1. what changed?
git diff -w <file>          # 2. review it
git add <file>              # 3. stage named files
git status                  # 4. only what you expect?
git commit -m "What changed"
git push
```

### OBSTACLES AND FIXES 5

| PROBLEM | CAUSE | FIX | LESSON |
|---|---|---|---|
| `**` and `#` visible on GitHub | Markdown escaped as `\*\*` by copying formatted text | Replaced the file using the code block's Copy button | Copy code blocks with the Copy button |
| `**What:**The` showed asterisks | No space after the closing `**` | Added the space | Always a space after closing bold |
| Commit message contained a placeholder | Example text copied as is | `git commit --amend` + `git push --force-with-lease` | Only rewrite your own latest commit |
| README code boxes broke the numbered list | Fences not indented; VS Code auto-closing added backticks | Commands as inline code instead | Prefer the format that cannot break |
| Status and run steps out of date | Docs written before automation existed | Updated README and ETL guide after the pipeline passed | Documentation follows the build, never ahead of it |

## Where each data issue was solved

Profiling decided the treatment; the transform layers apply it. Each code (T1–T9) comes from the decision log in `docs/02_data_quality_findings.md`.

| # | ISSUE | SIZE | TREATMENT | APPLIED IN |
|---|---|---|---|---|
| T1 | Incomplete 2016; Sep–Oct 2018 cut-off | period | Flag `in_analysis_period` (Jan 2017 – Aug 2018) | `stg_orders` `dim_date` |
| T2 | `customer_id` is per order | 96,096 people | Identify people by `customer_unique_id` | `stg_customers` `dim_customer` |
| T3 | Delivered without delivery date | 8 orders | Flag `excl_delivery_time` | `stg_orders` |

| | | | | |
|---|---|---|---|---|
| T4 | Carrier pickup before purchase | 166 orders | Flag `excl_carrier_leg` | `stg_orders` |
| T5 | Shipping deadline > 30 days after purchase | 239 items | Flag `excl_handover` | `fact_order_items` `fact_orders` |
| T6 | Several reviews per order | 547 orders | Flag latest review; facts use it | `stg_order_reviews` `fact_orders` |
| T7 | Order without review | 768 orders | `has_review`; score stays NULL | `fact_orders` |
| T8 | Product without category | 610 products | Label `unknown` | `stg_products` |
| T9 | Category without English name | 2 categories | Manual translation rows | `stg_category_translation` |

## Tools and where each was used

| TOOL | VERSION | ROLE | STAGES |
|---|---|---|---|
| PowerShell | Windows | Run Python, Git and the pipeline | All |
| Python | 3.14.7 | Loader and orchestrator | 2, 6 |
| pandas | 3.0.6 | Read CSV files | 2 |
| SQLAlchemy | 2.1.4 | Write DataFrames to PostgreSQL | 2 |
| psycopg | 3.3.6 | PostgreSQL driver; runs SQL files in the pipeline | 2, 6 |
| python-dotenv | — | Read connection settings from `.env` | 2, 6 |
| PostgreSQL | 18.6 | Database; runs every transform | 2–6 |
| DBeaver | — | Explore tables, run profiling | 3 |
| VS Code + PostgreSQL extension | — | Write files, run SQL, terminal, Markdown preview | 3–7 |
| Git / GitHub | 2.56.0 | Version control and portfolio hosting | All |
| Markdown / Mermaid | — | Documentation and diagrams | 3, 5, 7 |

## Principles the whole pipeline follows

### Reproducible

Everything rebuilds from the source files with one command.

### Idempotent

Every script drops and rebuilds its object; reruns are safe.

### Fail loudly

Bad values, wrong grain and failed checks stop the run.

### Flags, not deletion

Excluded rows stay in the data, marked per analysis.

### One place for each rule

Regions, translations and treatments are defined once.

### Verify with numbers

Each layer reconciles to figures checked in profiling.

## Run it

```
cd D:\portfolio\olist-commercial-analytics
.venv\Scripts\Activate.ps1
py run_pipeline.py              # full rebuild from the CSV files
py run_pipeline.py --skip-load  # rebuild staging and marts only
echo $LASTEXITCODE              # 0 = success, 1 = a step or check failed
```

Built during Weeks 1–2 of Project 1 (Olist commercial analytics). Numbers come from the profiling queries, the decision log and the pipeline checks in the repository `dbsaragih/olist-commercial-analytics`. Next stage: analysis of BQ1–BQ5 on the star schema, then the Power BI dashboard.